MACRO-AWARE STOCK FORECASTING USING BI-GRU ENSEMBLE AND FINBERT SENTIMENT INTEGRATION

A project report submitted in partial fulfillment of the requirements for the award of the degree of

MASTERS OF COMPUTER SCIENCE

By

ARAVIND P **REG NO. DVAYMCS003**

Under the Guidance of

Ms. ANJANA TK

DEPARTMENT OF COMPUTER SCIENCE

ST.JOSEPH'S COLLEGE (AUTONOMOUS), DEVAGIRI,CALICUT

MARCH 2026

ST.JOSEPH'S COLLEGE (AUTONOMOUS)
DEVAGIRI, CALICUT

Macro-Aware Stock Forecasting Using Bi-GRU Ensemble and FinBERT Sentiment Integration

Report of the project submitted to St.Joseph's College(Autonomous) Devagiri,Calicut
Affiliated to the University of Calicut

ARAVIND P **REG NO. DVAYMCS003**

Programme: Master of Computer Science

Semester: IV

DEPARTMENT OF COMPUTER SCIENCE

March 2026

Principal Head of the Department Supervisor

College seal

CERTIFICATE

This is to certify that the project report titled

MACRO-AWARE STOCK FORECASTING USING BI-GRU ENSEMBLE AND FINBERT SENTIMENT INTEGRATION

Submitted by

ARAVIND P **(REG NO. DVAYMCS003)**

St. Joseph's College (Autonomous), Devagiri, Calicut in partial fulfillment of the requirements for the award of degree of Master of Computer Science during the year 2024-2026.

Internal Examiner External Examiner

DECLARATION

I hereby declare that the project entitled **"Macro-Aware Stock Forecasting Using Bi-GRU Ensemble and FinBERT Sentiment Integration"** has been undertaken by me for the award of the degree of Master of Computer Science. I have completed this project under the guidance of Ms. Anjana T.K, Assistant Professor, Department of Computer Science, St. Joseph's College (Autonomous), Devagiri, Calicut.

Place: Calicut

Date:

ARAVIND P **REG NO. DVAYMCS003**

CERTIFICATE

This is to certify that the project report titled **"Macro-Aware Stock Forecasting Using Bi-GRU Ensemble and FinBERT Sentiment Integration "** record of project work done by him during the academic year 2024 – 2026 under my guidance and supervision in partial fulfilment of the requirement of degree of Master of Computer Science.

Place: Devagiri

Date: Ms. Anjana T.K

ACKNOWLEDGEMENT

I wish to express my sincere gratitude to **Fr. Dr. Biju Joseph**, principal of St. Joseph's College(Autonomous), Devagiri, Calicut for providing necessary facilities for developing my project.

I am profoundly thankful to **Dr. Anima P**, Head of the Department and my internal guide, for her continuous support, guidance, and constructive suggestions during the course of this project

I am also grateful to **Ms. Anjana T.K**, Assistant Professor as well as my internal guide, for her immense support and valuable suggestions

ARAVIND P **REG NO. DVAYMCS003**

ABSTRACT

ABSTRACT

The rapid advancement of financial markets driven by macroeconomic volatility, investor sentiment, and cross-domain market dependencies has made accurate short-term stock price forecasting an increasingly challenging problem. Conventional statistical models such as ARIMA and linear regression fail to capture nonlinear temporal dependencies, while most deep learning-based systems rely solely on historical price data without integrating macroeconomic awareness or structured sentiment intelligence.

To address these limitations, this work proposes a Macro-Aware Stock Forecasting Framework using a Bidirectional Gated Recurrent Unit (Bi-GRU) Ensemble integrated with FinBERT-Based Sentiment Intelligence and macroeconomic exchange rate signals. The proposed system models short-term return dynamics by fusing three complementary intelligence layers: technical price-based indicators, USD/INR macroeconomic signals, and structured financial news sentiment features extracted using FinBERT.

A multi-source sentiment pipeline was constructed using historical financial news datasets. Sentiment polarity was extracted using FinBERT and aggregated at both stock-level and macro-level, followed by normalization and clamping within the range [-1, 1] to ensure training stability. Engineered sentiment features include seven-day rolling sentiment averages, sentiment momentum, news shock indicators, and article frequency metrics. These features were fused with technical indicators such as RSI, SMA-50, and 20-day volatility metrics along with USD/INR macroeconomic signals to form a comprehensive ten-dimensional feature vector.

The forecasting architecture consists of a stacked Bidirectional GRU layer with L2 regularization, dropout stabilization, and a three-model ensemble averaging strategy to reduce variance and improve generalization. A transductive K-Nearest Neighbour (KNN) refinement mechanism was further applied to adjust predictions based on historical regime similarity using cosine distance. The system was optimized using a custom profit-focused loss function that penalizes directional misclassification and large-magnitude prediction errors with adaptive weighting.

TABLE OF CONTENTS

	Page No
CERTIFICATE 1	II
CERTIFICATE 2	III
DECLARATION	IV
CERTIFICATE 3	V
ACKNOWLEDGEMENT	VI
ABSTRACT	VII
TABLE OF CONTENTS	IX
CHAPTER I INTRODUCTION	1
CHAPTER II LITERATURE SURVEY	5
CHAPTER III	
PROPOSED SYSTEM AND METHODOLOGY	
3.1 PROPOSED SYSTEM	13
3.2 METHODOLOGY	14

CHAPTER IV

TRAINING ,TESTING AND VALIDATION 20

CHAPTER V

IMPLEMENTATION AND RESULTS

5.1 IMPLEMENTATION 27

5.2 WEB SERVICES AND FRONTEND DEVELOPMENT 29

CHAPTER VI

CONCLUSION AND FUTURE ENHANCEMENT

6.1 CONLCUSION 37

6.2 FUTURE ENHANCEMENT 38

REFERENCES 39

APPENDIX A : RESULT AND SCREENSHOT 42

APPENDIX B : ABBREVIATIONS 46

CHAPTER I

INTRODUCTION

CHAPTER I
INTRODUCTION

Financial markets are complex, dynamic, and highly nonlinear systems influenced by a wide range of economic, political, and psychological factors. Accurate stock price forecasting has long been a fundamental objective in financial research, algorithmic trading, portfolio optimization, and risk management. Despite decades of study, predicting short-term stock movements remains challenging due to volatility clustering, regime shifts, market sentiment fluctuations, and macroeconomic dependencies. The enormous volume of financial transactions occurring daily, combined with the speed at which new information is absorbed by the market, makes it virtually impossible for human analysts to process and act on all available signals simultaneously. As global markets become increasingly interconnected, the influence of cross-border capital flows, currency fluctuations, and geopolitical developments on individual equity prices has grown substantially, demanding more sophisticated and context-aware forecasting systems.

Traditional forecasting approaches such as autoregressive integrated moving average (ARIMA), linear regression, and econometric factor models rely on assumptions of stationarity and linear relationships. While these models provide interpretability, they often fail to capture nonlinear temporal dependencies and interactions between heterogeneous financial variables. With the advancement of deep learning, recurrent neural networks (RNNs), Long Short-Term Memory (LSTM) networks, and Gated Recurrent Units (GRUs) have demonstrated superior ability in modelling sequential time-series behaviour. However, many existing deep learning-based stock prediction models primarily focus on historical price data and technical indicators, neglecting macroeconomic signals and structured sentiment intelligence, which are critical drivers of real-world market movements. These limitations result in models that perform well under stable market conditions but degrade significantly during macroeconomic shocks or news-driven volatility events.

In real-world financial markets, price movements are not solely driven by historical price patterns. They are significantly influenced by macroeconomic variables such as exchange rates and capital flows, global financial conditions, corporate and geopolitical news, and investor sentiment and behavioural reactions. The USD/INR exchange rate,

for instance, directly affects foreign institutional investment flows, corporate earnings in export-oriented sectors, and overall market risk perception in emerging economies. A depreciation of the Indian rupee against the US dollar can trigger capital outflows from equity markets, increasing selling pressure across multiple sectors simultaneously. Failure to integrate these cross-domain signals limits the robustness and generalization ability of forecasting systems across diverse market regimes and economic cycles.

Another critical limitation in conventional stock prediction models is the direct forecasting of raw stock prices. Since financial time series exhibit heteroskedasticity and multiplicative growth behaviour, modelling raw prices often leads to unstable learning and biased predictions. Log-return forecasting provides a statistically sound alternative by stabilizing variance and transforming multiplicative price changes into additive structures, thereby improving neural network convergence and predictive stability across multiple assets with vastly different price scales. This transformation also ensures stationarity properties that are essential for reliable sequence modelling in recurrent architectures.

In recent years, Natural Language Processing (NLP) techniques have enabled sentiment-aware financial modelling. Transformer-based language models such as FinBERT have been specifically fine-tuned for financial text classification, allowing structured extraction of positive, negative, and neutral sentiment signals from financial news articles. However, many sentiment-based forecasting systems suffer from poor aggregation strategies, noise sensitivity, and lack of macro-level integration, limiting their practical effectiveness in multi-asset forecasting pipelines. Most existing systems use raw sentiment polarity scores as static inputs without engineering temporal features such as rolling averages, sentiment momentum, or news shock intensity, which are necessary to capture persistent and evolving investor behavioural trends.

To address these limitations, this project proposes a Macro-Aware Stock Forecasting Using Bi-GRU Ensemble with FinBERT Sentiment Integration. The system integrates three major intelligence layers: technical price-based indicators, macroeconomic exchange rate signals (USD/INR), and financial news sentiment intelligence. The proposed architecture introduces log-return-based forecasting for statistical stability, RobustScaler-based normalization to handle fat-tailed distributions, a stacked Bidirectional GRU architecture for enhanced temporal modelling, three-model ensemble averaging to reduce prediction variance, transductive K-Nearest Neighbour refinement for regime-based correction, and a custom profit-focused loss function

emphasizing directional accuracy and significant market movements.

The primary objectives of this project are to develop a stable deep learning framework for short-term stock return forecasting, to integrate macroeconomic awareness into sequence-based neural architectures, to construct a structured financial sentiment intelligence pipeline using FinBERT, to improve directional prediction accuracy using profit-focused optimization, and to evaluate ensemble stabilization and transductive refinement in financial time-series prediction. The system was evaluated across 58 publicly traded equities on the National Stock Exchange (NSE), demonstrating strong generalization across both stable and volatile market regimes. The remainder of this report presents the literature survey, system architecture, methodological framework, experimental validation, performance analysis, and future research directions of the proposed macro-aware Bi-GRU ensemble forecasting system.

Stock market forecasting has evolved significantly over the past decade, transitioning from rule-based statistical approaches to data-driven deep learning architectures capable of processing multiple heterogeneous information streams simultaneously. The increasing availability of financial news data, social media sentiment, macroeconomic indicators, and high-frequency price data has opened new opportunities for building more comprehensive and robust forecasting systems. The proposed framework in this project leverages these advancements by combining recurrent deep learning with transformer-based sentiment modelling and macroeconomic signal integration, establishing a unified pipeline that is both statistically grounded and financially meaningful across diverse equity assets.

Unlike conventional single-asset models that operate in isolation, the proposed system is designed to capture cross-asset dependencies and sector-level correlations, enabling a more holistic understanding of market dynamics. By integrating macroeconomic variables such as USD/INR exchange rate, crude oil prices, and gold prices alongside technical indicators and FinBERT-derived sentiment scores, the model ensures that both external economic conditions and investor psychology are reflected in the final price forecasts. This multi-dimensional approach positions the framework as a scalable and interpretable solution for real-world equity forecasting on the National Stock Exchange of India.

CHAPTER II

LITERATURE SURVEY

CHAPTER II
LITERATURE SURVEY

Stock market prediction has become a critical research area due to the rapid advancement of deep learning architectures, transformer-based language models, and the increasing availability of multi-source financial data. As forecasting models become increasingly sophisticated, numerous approaches have been proposed, each addressing different weaknesses in capturing temporal dependencies, sentiment dynamics, and macroeconomic influences. This literature review synthesizes twelve significant research contributions spanning from 2019 to 2024, highlighting their methodologies, strengths, and limitations.

Peivandizadeh et al. (2024) proposed a hybrid forecasting framework that integrates social media sentiment analysis with a Transductive Long Short-Term Memory (TLSTM) model for stock market prediction. A major challenge addressed in this work is the uneven distribution of sentiment categories in social media data, where neutral sentiments dominate over positive and negative ones. To tackle this class imbalance, the authors introduced an Off-Policy Proximal Policy Optimization (PPO) algorithm that adjusts the reward mechanism during training, favouring correct classification of minority sentiment classes. In the second stage, the TLSTM model integrates sentiment outputs with historical OHLCV stock data, dynamically weighting training samples based on their proximity to the test point to enhance temporal sensitivity. The framework was evaluated on 50 NSE stocks and 10 ETFs using RMSE, MAPE, and MAE metrics, achieving substantial improvements over conventional LSTM and machine learning baselines. However, the study lacks macroeconomic variable integration such as exchange rate indicators, depends on a single TLSTM model without ensemble-based stabilization, directly predicts raw stock prices instead of statistically stable log-returns.

Dong et al. (2024) proposed a FinBERT-LSTM hybrid framework for stock price prediction that combined transformer-based financial sentiment embeddings with LSTM-based sequence modelling. The authors extracted structured sentiment signals from financial news articles using FinBERT and fed them as auxiliary inputs alongside

historical OHLCV price data into the LSTM forecasting module. Experimental evaluations on technology-sector stocks demonstrated improved prediction accuracy compared to price-only LSTM baselines, confirming the value of domain-specific sentiment embeddings in financial forecasting. However, the framework did not incorporate macroeconomic indicators such as exchange rate variables, relied on a single LSTM model without ensemble variance reduction, and lacked engineered temporal sentiment features such as rolling averages and sentiment momentum, limiting its ability to capture persistent and evolving investor sentiment trends across multiple assets.

Shah et al. (2024) conducted a comparative analysis of LSTM, GRU, and Transformer models for financial time-series prediction across multiple market indices. The study evaluated model performance using MAE, RMSE, and directional accuracy metrics on daily closing price data sourced from major stock exchanges. Results indicated that GRU-based models achieved competitive performance with significantly lower computational cost compared to Transformer architectures, confirming their efficiency for financial sequence modelling tasks. The study highlighted that GRU networks effectively captured short-to-medium range temporal dependencies in price series. However, none of the evaluated models incorporated macroeconomic variables, financial sentiment intelligence, or ensemble-based stabilization strategies, limiting their generalization ability across heterogeneous asset classes operating under diverse macroeconomic stress conditions.

Sivadasan et al. (2024) introduced a hybrid GRU-LSTM architecture that combined the computational efficiency of Gated Recurrent Units with the memory retention strength of LSTM networks for stock market forecasting. The hybrid framework applied a sliding window method to examine daily stock metrics and integrated various technical indicators to improve predictive refinement. Experiments on NSE equity datasets demonstrated improved training convergence and predictive accuracy compared to standalone GRU or LSTM models. Although the hybrid architecture showed improved temporal modelling performance, it remained restricted to numerical stock features and technical indicators. The framework did not incorporate macroeconomic awareness or financial news sentiment intelligence, which are crucial for modelling short-term behavioural market reactions during external information .

Behura et al. (2023) proposed a multi-layered sequential LSTM model for stock price forecasting, introducing the Adam optimizer with normalized time-interval data to capture hierarchical temporal dependencies and overcome the vanishing gradient problem common in standard neural networks. The stacked LSTM architecture aimed to enhance memory retention and long-term dependency learning by deepening the recurrent structure across multiple layers. Experimental results on benchmark equity datasets demonstrated improved prediction accuracy compared to shallow single-layer LSTM networks. However, the study primarily focused on historical price-based indicators and technical analysis features alone. The absence of macroeconomic variables and structured sentiment integration limited the model's capability to capture external market influences from news events and currency fluctuations.

Chen et al. (2023) introduced a multi-feature stock price prediction model combining 57 technical indicators with LASSO feature selection and a Cascade LSTM (Ca-LSTM) network to extract more nuanced information from financial time-series data. The LASSO algorithm identified the most relevant features from a large candidate set, reducing dimensionality and improving training efficiency. The Ca-LSTM architecture enabled hierarchical feature extraction across cascaded recurrent layers, improving short-term prediction reliability. Despite its comprehensive technical feature engineering, the framework did not integrate macroeconomic exchange rate variables or financial news sentiment signals, and lacked ensemble stabilization mechanisms to reduce stochastic variance across training runs, limiting its robustness during volatile market regimes.

Wu et al. (2022) presented SI-LSTM, a stock price prediction model that fused multiple data sources including historical stock information, technical indicators, financial news, and stock forum discussions within a unified LSTM framework. The model employed convolutional neural networks for sentiment feature extraction from textual data, which was subsequently fused with numerical price features and fed into LSTM layers for prediction on the China Shanghai A-share market. The study demonstrated that multi-source feature integration consistently outperformed single-source baselines.

Ye et al. (2022) proposed a stacking ensemble deep learning model for cryptocurrency price prediction that combined LSTM and GRU neural networks with a stacking ensemble technique to improve forecasting accuracy. The model incorporated price data, technical indicators, and sentiment indices derived from social media text using linguistic and statistical analysis methods. The stacking ensemble mechanism reduced individual model variance and improved generalization across different market periods. While the ensemble strategy and sentiment integration showed measurable improvements, the framework was evaluated exclusively on cryptocurrency price prediction rather than multi-asset equity forecasting, did not incorporate macroeconomic variables, and relied on general-purpose sentiment methods rather than domain-specific financial NLP models such as FinBERT.

Li et al. (2022) proposed a hybrid model combining attention mechanisms with Bidirectional GRU networks for multi-step stock price forecasting across multiple exchange-listed equities. The attention layer was designed to assign differential importance weights to temporal steps within the input sequence, enabling the model to focus on the most financially significant time periods during prediction. Experiments demonstrated that the attention-enhanced Bi-GRU model outperformed standard GRU and LSTM baselines in both regression accuracy and temporal pattern recognition. However, the study did not integrate external macroeconomic signals or sentiment-based features into the feature vector, and lacked ensemble stabilization strategies to reduce stochastic training variance across different weight initialization runs, which is critical for production-level multi-asset forecasting pipelines.

Abdelfattah et al. (2022) proposed a unique approach to improve stock market movement predictions by refining sentiment analysis with neutrosophic logic, which adeptly manages uncertain and indeterminate data from social media sources. This method involved classifying social media sentiments more accurately and using these insights along with historical stock data as inputs for a deep learning LSTM model to forecast stock movements over a defined period. The neutrosophic logic framework provided a more nuanced representation of sentiment uncertainty compared to binary polarity classification. The framework relied solely on social media text without incorporating financial news datasets processed through domain-specific models, did not include macroeconomic exchange rate features.

Araci (2019) proposed FinBERT, a domain-specific adaptation of BERT fine-tuned on financial communication datasets including earnings call transcripts, financial news articles, and analyst reports for sentiment classification. FinBERT demonstrated superior performance in classifying financial text into positive, negative, and neutral sentiment categories compared to general-purpose NLP models such as VADER and standard BERT. The model established a new benchmark for financial sentiment analysis by learning domain-specific vocabulary, financial abbreviations, and market-related linguistic patterns. While FinBERT significantly improved sentiment extraction accuracy, most subsequent forecasting frameworks adopted its outputs only as static categorical polarity indicators, without engineering dynamic temporal sentiment features such as rolling averages, sentiment momentum, and news shock intensity measures.

Devlin et al. (2019) introduced BERT, a transformer-based language model pre-trained using masked language modelling and next sentence prediction objectives on large-scale text corpora. BERT significantly improved contextual understanding in natural language processing tasks by learning deep bidirectional relationships between words and their surrounding context, enabling rich semantic embedding generation across diverse downstream tasks. The transformer architecture enabled the creation of contextual embeddings that became foundational for domain-specific sentiment modelling in financial analytics and other specialized fields. Although BERT provided a powerful general-purpose language understanding framework, its direct application to financial text was constrained by the specialized vocabulary, numerical expressions, and domain-specific jargon present in financial documents, which subsequently motivated the development of domain-adapted variants specifically fine-tuned for financial language understanding.

Identified Research Gaps

The reviewed literature illustrates the rapid progression of stock forecasting research, evolving from traditional statistical models to hybrid deep learning architectures integrating sentiment intelligence. Although these approaches demonstrate strong performance within controlled experimental settings, they consistently exhibit the following common limitations:

1. **Lack of macroeconomic variable integration** — Most existing frameworks rely solely on price-based technical indicators and neglect exchange rate signals such as USD/INR, which significantly influence emerging market equity returns through foreign capital flow dynamics.
2. **Absence of structured temporal sentiment engineering** — Existing sentiment-aware models use raw FinBERT polarity scores as static inputs without deriving engineered temporal features such as rolling sentiment averages, sentiment momentum, or news shock intensity indicators.
3. **Dependence on single-model architectures** — Most frameworks rely on a single trained model without ensemble-based variance reduction, making predictions sensitive to stochastic weight initialization and prone to instability during volatile regimes.
4. **Raw price prediction instead of log-return modelling** — Direct prediction of closing prices on heterogeneous multi-asset datasets leads to unstable training due to scale differences and heteroskedasticity, which log-return transformation resolves.
5. **No regime-aware transductive refinement** — Existing models apply uniform inference across all market regimes without adjusting predictions based on historical regime similarity, reducing robustness during structurally shifting market conditions.
6. **Absence of financially aligned optimization objectives** — Standard regression loss functions such as MSE do not account for large-magnitude market movements, which are the primary concerns in financial forecasting pipelines.

These identified gaps collectively motivate the development of the proposed macro-aware, ensemble-based Bi-GRU framework with FinBERT-driven sentiment intelligence, log-return forecasting, and profit-focused optimization presented in this work.

CHAPTER III

PROPOSED SYSTEM AND METHODOLOGY

CHAPTER III
PROPOSED SYSTEM AND METHODOLOGY

3.1 PROPOSED SYSTEM

The proposed system presents a Macro-Aware Stock Price Forecasting Framework based on a Bidirectional Gated Recurrent Unit (Bi-GRU) ensemble integrated with FinBERT-based financial sentiment intelligence and macroeconomic awareness. The system is designed to model short-term market dynamics by fusing heterogeneous data sources, including historical price signals, engineered technical indicators, macroeconomic exchange rate variables, and structured textual sentiment information extracted from financial news articles. Unlike conventional stock forecasting models that rely exclusively on autoregressive price patterns, the proposed framework introduces a multi-layer information fusion strategy that captures nonlinear temporal dependencies while incorporating behavioural and macroeconomic signals that significantly influence short-term return volatility.

The model forecasts next-day log-returns instead of raw stock prices to ensure statistical stability, variance normalization, and additive temporal representation. Ensemble stabilization and transductive regime refinement are incorporated to reduce predictive variance and enhance robustness during volatile market regimes. Each stock is processed independently through the pipeline to preserve asset-specific behavioural patterns and avoid cross-contamination of features across different equities.

The complete system operates in a modular pipeline consisting of the following stages:

1. Data Acquisition and Synchronization
2. Sentiment Intelligence Construction
3. Feature Engineering and Normalization
4. Sequential Framing and Window Construction
5. Bi-GRU Ensemble Forecasting
6. Transductive Regime Refinement
7. Price Reconstruction and Evaluation

3.2 METHODOLOGY

3.2.1 Data Acquisition and Synchronization

Stock Market Data

Daily historical OHLCV data (Open, High, Low, Close, Volume) are collected for multiple publicly traded equities listed on the National Stock Exchange (NSE) of India. From these values, the primary modelling variable, log-return, is computed as:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

Log-return modelling offers several advantages over raw price prediction. It stabilizes variance across time, converts multiplicative price changes into additive form, improves stationarity properties of the time series, enhances neural network convergence stability, and reduces scale sensitivity across multiple assets with widely different price levels. The target variable is defined as the next-day log-return:

$$Target_Log_Return_t = r_{t+1}$$

Macroeconomic Signal Integration

To introduce macroeconomic awareness into the forecasting pipeline, the USD/INR exchange rate is incorporated as an external explanatory variable. Exchange rate fluctuations influence foreign institutional investment flows, corporate earnings especially in export and import sectors, risk perception in emerging markets, and cross-border capital movement patterns. The macroeconomic data are temporally aligned with stock trading dates using forward-fill interpolation for non-trading days to maintain synchronization across the complete dataset.

3.2.2 Financial News and Sentiment Extraction

Financial news articles are processed through FinBERT, a domain-specific transformer model fine-tuned for financial sentiment classification, to extract sentiment polarity probabilities: P(Positive), P(Negative), and P(Neutral). A continuous sentiment score is computed as:

$$Sentiment = P(Positive) - P(Negative)$$

To prevent extreme polarity distortion from outlier articles, sentiment values are clamped within the range [-1, 1]. This transformation ensures numerical stability and prevents gradient explosion during model training across all 58 assets in the pipeline.

3.2.3 Sentiment Intelligence Engineering

Instead of using raw sentiment classification outputs as static inputs, the system constructs dynamic sentiment indicators that capture temporal behavioural trends.

The following five engineered sentiment features are derived for each stock:

Stock_Sentiment_7D — Seven-day rolling average of stock-specific sentiment scores to smooth short-term noise and capture persistent investor perception trends over the trading week.

Macro Sentiment 7D — Seven-day rolling average of aggregated market-wide sentiment to represent the broader macroeconomic mood affecting all equities simultaneously.

Sentiment Momentum — First-order difference of the rolling sentiment average to capture acceleration or deceleration in sentiment trends over consecutive trading days:

$$Sentiment_Momentum_t = S_t - S_{t-1}$$

News Shock — Absolute deviation of daily sentiment from a rolling baseline to detect abnormal information shocks caused by unexpected corporate or geopolitical events.

3.2.4 Feature Engineering and Normalization

The final feature vector for each stock consists of ten dimensions representing a comprehensive combination of price dynamics, momentum indicators, macroeconomic signals, and sentiment intelligence:

- Close
- Log_Returns
- RSI (Relative Strength Index)
- SMA_50 (50-day Simple Moving Average)
- Volatility_20D (20-day rolling standard deviation)
- USD_INR
- Macro_Sentiment_7D
- Stock_Sentiment_7D
- Sentiment_Momentum
- News_Shock

Technical indicators such as RSI and SMA_50 capture momentum and trend persistence respectively, while Volatility_20D measures risk clustering patterns. To mitigate the effect of heavy-tailed financial distributions commonly observed in equity return data, RobustScaler is applied for feature normalization:

$$X_{scaled} = \frac{X - Median(X)}{IQR(X)}$$

This scaling approach reduces sensitivity to extreme outliers compared to MinMax or Standard scaling methods, making it particularly appropriate for financial time-series data with frequent volatility spikes. The scaler is fitted exclusively on the training dataset and subsequently applied to the test dataset to prevent data leakage during evaluation.

3.2.5 Sequential Window Construction

A sliding window mechanism with a lookback period of 15 trading days is applied to convert tabular time-series data into three-dimensional tensors suitable for recurrent learning. Each input sequence represents the previous 15 trading days of multi-dimensional features used to predict the next-day log-return. The input tensor shape for each sample is (15, 10), representing 15 time steps across 10 feature dimensions. This framing enables the model to capture short-term memory effects, volatility clustering, sentiment drift patterns, and local regime structures within each asset's historical trajectory.

3.2.6 Bi-GRU Forecasting Architecture

The core forecasting module consists of a stacked Bidirectional GRU architecture with four functional layers:

Layer 1 — Bidirectional GRU (64 Units): Captures both forward and backward temporal dependencies within the 15-day input window. L2 regularization with $\lambda = 0.001$ is applied to prevent overfitting during per-stock independent training runs.

Layer 2 — Dropout (0.25): Reduces co-adaptation of neurons during training and enhances generalization to unseen test sequences by randomly deactivating 25% of neurons per forward pass.

Layer 3 — GRU (32 Units): Extracts condensed latent representations from the bidirectional output, compressing the temporal information into a lower-dimensional feature space suitable for regression output.

Layer 4 — Dense Output Layer (1 Unit): Generates the predicted next-day log-return as a scalar regression output. The network is optimized using the Nadam optimizer, which combines adaptive gradient updates from Adam with Nesterov momentum acceleration for improved convergence stability.

3.2.7 Profit-Focused Loss Function

A custom loss function is designed to align model optimization with financial trading objectives rather than purely statistical accuracy. The loss penalizes directional misclassification with a weight of 5×, large-magnitude prediction errors, and high-volatility movement errors with a scaling factor of 2× for absolute returns exceeding 1%. Mathematically:

$$Loss = (direction_error \times 5 + 1) \times | y_{true} - y_{pred} | \times size_weight$$

where size_weight = 2.0 when |return| > 0.01, and 1.0 otherwise. This design improves sensitivity to significant market moves that carry the greatest financial consequence for trading decisions.

3.2.8 Ensemble Stabilization Strategy

To reduce prediction variance arising from stochastic weight initialization, three independently initialized Bi-GRU models are trained for each stock. The final ensemble prediction is computed as the simple average across all three models:

$$\hat{r}_t = \frac{1}{3}\sum_{i=1}^{3} \hat{r}_t^{(i)}$$

Ensemble learning reduces sensitivity to random initialization variability and improves prediction stability across volatile market regimes where individual models may exhibit divergent behaviour.

3.2.9 Transductive Regime Refinement

Post-ensemble predictions are further refined using a transductive K-Nearest Neighbour (KNN) mechanism that introduces local regime awareness into the inference process. The procedure involves flattening input sequence representations, computing cosine similarity between test sequences and all training sequences, identifying the 20 nearest historical patterns, computing the mean neighbour return, and blending the model prediction with the neighbour-based estimate:

$$\hat{r}_{final} = 0.7 \cdot \hat{r}_{model} + 0.3 \cdot \hat{r}_{neighbour}$$

This mechanism introduces explicit regime awareness without modifying recurrent network parameters during inference, providing computational efficiency compared to internal transductive adaptation approaches used in prior work.

3.2.10 Price Reconstruction

Predicted log-returns are converted back to absolute stock prices using the exponential transformation to ensure financial consistency and preserve multiplicative price dynamics:

$$P_t = P_{t-1} \cdot e^{\hat{r}_t}$$

This reconstruction step enables direct comparison of predicted prices against actual closing prices using standard regression evaluation metrics including MAPE, RMSE, MAE, and R^2.

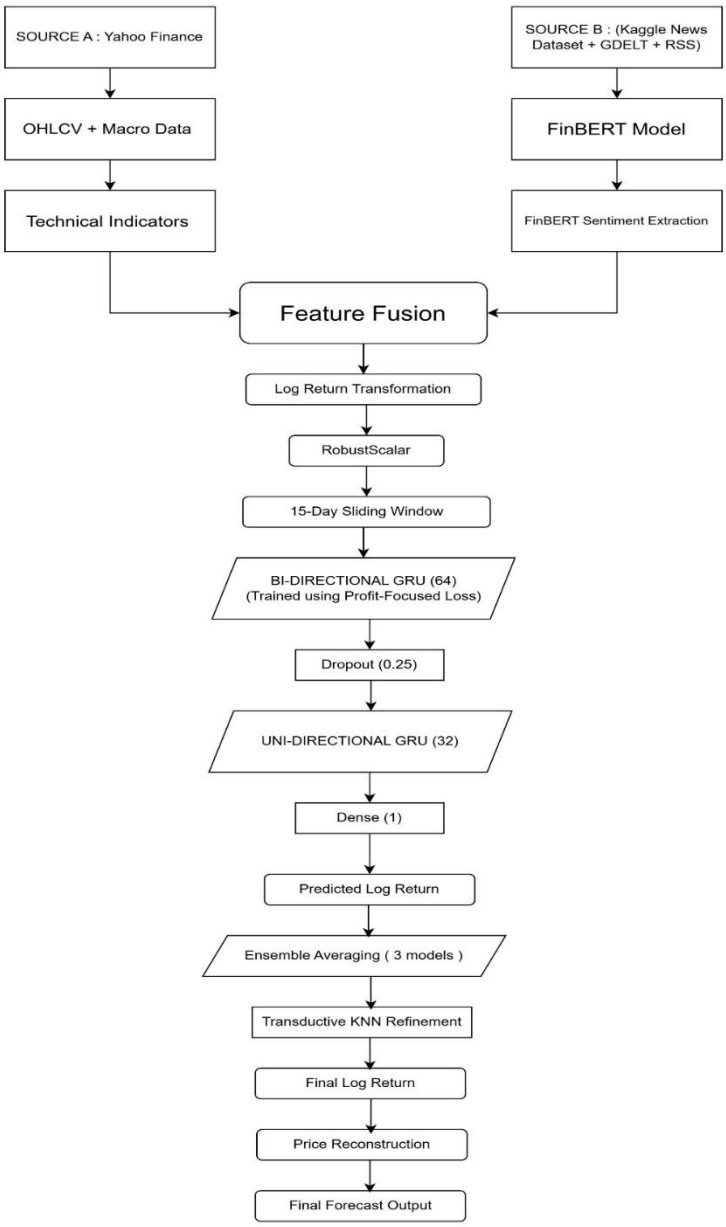


Fig 3.1: Workflow Diagram of the Proposed Macro-Aware Bi-GRU Ensemble Framework

3.2.12 Training Hyperparameters

The selection of training hyperparameters plays a crucial role in ensuring stable convergence and effective learning in recurrent architectures like the proposed Bi-GRU ensemble. Each stock is trained independently to preserve asset-specific behavioural dynamics. The complete set of hyperparameters used across all 58 assets is listed below:

Component	**Value**
Lookback Window	15 days
Train / Test Split	85% / 15% (chronological)
Validation Split	20% of training set
Ensemble Size	3 models
Bi-GRU Units	64
GRU Units	32
Dropout Rate	0.25
L2 Regularization	0.001
Optimizer	Nadam
Batch Size	32
Maximum Epochs	25
Early Stopping Patience	8
Minimum Row Threshold	200 rows

A chronological train-test split is strictly enforced to prevent data leakage, ensuring that no future information influences training. Early stopping is implemented to terminate training when validation loss fails to improve for 8 consecutive epochs, preventing overfitting and reducing unnecessary computation. The RobustScaler is fitted exclusively on the training partition and applied to the test partition to maintain evaluation integrity. Each of the three ensemble models is initialized independently with different random seeds to maximize diversity in the learned weight configurations, ensuring that the ensemble averaging mechanism provides genuine variance reduction rather than redundant predictions.

3.2.13 Evaluation and Performance Metrics

To evaluate the performance and reliability of the proposed Macro-Aware Bi-GRU Ensemble framework, four standard regression metrics are used. These metrics quantify the model's ability to accurately forecast stock returns while revealing its strengths and behaviour across different volatility conditions.

Mean Absolute Percentage Error (MAPE): MAPE measures the average percentage deviation of predicted prices from actual prices. It provides a scale-independent measure of forecasting accuracy that is directly interpretable across assets with different price levels. Lower MAPE values indicate stronger predictive performance.

$$MAPE = \frac{1}{n}\sum_{t=1}^{n} \left| \frac{y_t - \hat{y}_t}{y_t} \right| \times 100$$

Root Mean Square Error (RMSE): RMSE measures the square root of the average squared prediction error, penalizing large deviations more heavily than small ones. It is particularly sensitive to outlier predictions during high-volatility market events, making it a useful indicator of worst-case forecasting behaviour.

$$RMSE = \sqrt{\frac{1}{n}\sum_{t=1}^{n} (y_t - \hat{y}_t)^2}$$

Mean Absolute Error (MAE): MAE measures the average absolute deviation between predicted and actual prices without squaring the errors, providing a more robust measure of central prediction accuracy that is less sensitive to extreme outliers compared to RMSE.

$$MAE = \frac{1}{n}\sum_{t=1}^{n} | y_t - \hat{y}_t |$$

Coefficient of Determination (R^2): R^2 measures the proportion of variance in actual prices that is explained by the model predictions. A value closer to 1.0 indicates stronger explanatory power and better overall model fit. To prevent metric distortion on low-variance assets, R^2 computation includes a safeguard that excludes assets with a coefficient of variation below 0.001.

$$R^2 = 1 - \frac{\sum_{t=1}^{n}(y_t - \hat{y}_t)^2}{\sum_{t=1}^{n}(y_t - \bar{y})^2}$$

Performance metrics are computed individually for each of the 58 stocks and subsequently averaged to assess the overall generalization capability of the framework across heterogeneous asset classes and market regimes.

The choice of these four complementary metrics ensures a comprehensive evaluation of the forecasting system from multiple perspectives. While MAPE provides an interpretable percentage-based accuracy measure suitable for comparing performance across assets with different price magnitudes, RMSE captures the severity of large prediction errors that carry the most financial risk. MAE offers a balanced central tendency measure that is robust to isolated spikes, while R^2 quantifies the overall explanatory strength of the model relative to a naive mean baseline. Together, these metrics provide a holistic picture of model performance across both stable low-volatility assets such as index ETFs and high-volatility individual equities with frequent sharp price movements.

All metric computations are performed on reconstructed absolute stock prices obtained through exponential transformation of predicted log-returns, ensuring that the evaluation reflects real-world financial accuracy rather than abstract statistical performance on normalized return values. The averaged metrics across all 58 assets provide a reliable estimate of the framework's generalization capability under diverse market conditions, confirming the robustness of the proposed macro-aware ensemble architecture across heterogeneous equity classes listed on the National Stock Exchange of India

CHAPTER IV

TRAINING, TESTING AND VALIDATION

CHAPTER IV

TRAINING, TESTING AND VALIDATION

The first rule of machine learning — never use the same dataset for model training and model evaluation. If you want to build a reliable machine learning model, you need to split your dataset into the training set, validation set, and test set. If you don't, your results will be biased and you will end up with a false impression of better model accuracy. In financial time-series forecasting systems, this principle is even more critical because temporal ordering must be strictly preserved. Unlike conventional machine learning tasks where data can be randomly shuffled, stock market data carries sequential dependencies where future values must never leak into the training process. Therefore, strict chronological splitting and structured validation protocols were adopted throughout this study to ensure unbiased and reliable performance measurement across all 58 assets.

4.1 Train, Validation and Test Set

For training and testing purposes of the proposed model, the complete dataset for each stock is broken down into three distinct partitions — the training set, validation set, and test set. The training set is used to optimize the model parameters, while the validation set guides hyperparameter tuning and prevents overfitting through early stopping.

4.1.1 Training Set

The training set is the portion of data used to train and make the model learn the hidden features and temporal patterns present in the financial time series. In each epoch, the same training data is fed to the Bi-GRU ensemble repeatedly, and the model continues to learn asset-specific dynamics including price momentum, sentiment trends, and macroeconomic dependencies. The training set should have a diversified set of market conditions so that the model is exposed to both bullish and bearish regimes, enabling it to generalize effectively to unseen future price movements. In the proposed framework, the training set comprises 85% of each stock's historical data, split chronologically to preserve temporal ordering and prevent any form of look-ahead bias during the learning process.

4.1.2 Validation Set

The validation set is a separate portion of data, drawn from within the training partition, used to monitor model performance during training. This validation process provides information that helps tune the model's hyperparameters and configurations accordingly. It acts as an internal benchmark telling the training process whether optimization is moving in the right direction or not. The model is trained on the training subset and simultaneously evaluated on the validation set after every epoch using the custom profit-focused loss function. The main purpose of maintaining a validation split is to prevent overfitting, where the model becomes highly accurate on training samples but fails to generalize to data it has not previously encountered. In the proposed framework, 20% of the training partition is reserved as the validation subset, resulting in an effective data distribution of approximately 68% training, 17% validation, and 15% final testing across each stock's complete historical dataset.

4.1.3 Test Set

The test set is a completely separate and untouched portion of data used to evaluate the model after training is fully completed. It provides an unbiased final performance measurement in terms of MAPE, RMSE, MAE, and R^2. No test data is used during training, validation, or feature scaling at any stage of the pipeline.

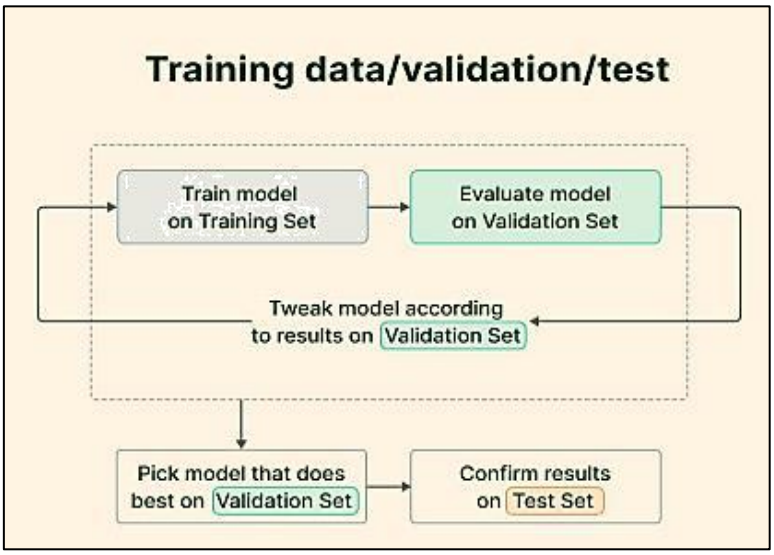


Fig 4.1: Training, Testing and Validation Set

4.2 Data Split Ratio

The train-validation-test split ratio in this study was determined based on two primary considerations: the total number of available daily records per stock and the sequential nature of financial time-series data. Since each stock is trained independently and the minimum row threshold is set at 200 records, the 85/15 chronological split ensures sufficient training data for the Bi-GRU ensemble to learn meaningful temporal patterns while reserving enough test samples for statistically reliable evaluation. The 20% internal validation split within the training partition provides adequate feedback for early stopping and hyperparameter monitoring without reducing the effective training window excessively. This results in an effective allocation of approximately 68% for training, 17% for validation, and 15% for testing, a well-justified and standard configuration widely adopted in financial deep learning literature. This split configuration ensures that the proposed framework is evaluated under realistic conditions that reflect actual deployment in live equity markets

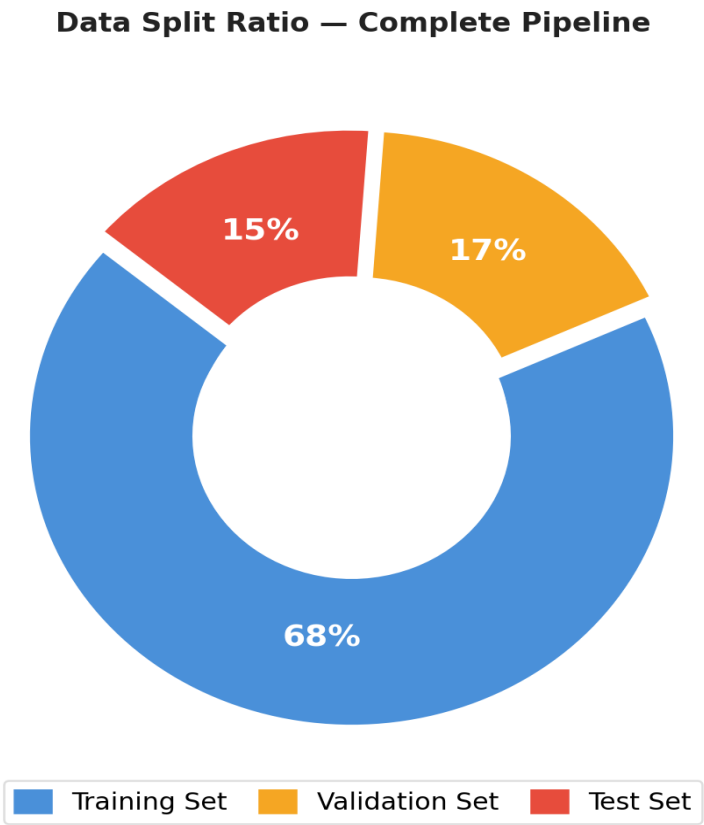


Fig 4.2: Data Split Ratio

CHAPTER V

IMPLEMENTATION AND RESULTS

CHAPTER V
IMPLEMENTATION AND RESULTS

5.1 Implementation

Python is commonly used for machine learning and financial analytics projects due to its versatility, extensive libraries, and ease of use. Its clean and readable syntax makes it accessible for developers of all levels of expertise. Python's large ecosystem of libraries and frameworks such as TensorFlow, Keras, Scikit-learn, Pandas, NumPy, and Streamlit provide powerful tools for building deep learning pipelines, processing financial time-series data, and deploying interactive forecasting interfaces. Furthermore, Python's strong community support ensures a wealth of resources, tutorials, and forums are available to assist developers. Python's scalability features and support for asynchronous programming enable efficient utilization of computational resources for building highly scalable machine learning applications.

Moreover, Python's dynamic typing, interpreted nature, and extensive standard library contribute to its suitability for rapid prototyping and development. This allows developers to quickly iterate on ideas, test concepts, and build minimum viable products efficiently. Overall, Python's simplicity, versatility, extensive libraries, and community support make it an excellent choice for machine learning projects, enabling developers to build scalable, robust, and efficient applications while minimizing development time and effort.

Python is an interpreted, high-level, general-purpose programming language created by Guido van Rossum and first released in 1991. Python's design philosophy emphasizes code readability with its notable use of significant whitespace. Its language constructs and object-oriented approach aim to help programmers write clear, logical code for small and large-scale projects. Python is dynamically typed and garbage-collected. It supports multiple programming paradigms, including procedural, object-oriented, and functional programming. Python is often described as a "batteries included" language due to its comprehensive standard library.

5.1.1 Python

Python is a multi-paradigm programming language. Object-oriented programming and structured programming are fully supported, and many of its features support functional

programming and aspect-oriented programming. Python uses dynamic typing and a combination of reference counting and a cycle-detecting garbage collector for memory management. It also features dynamic name resolution (late binding), which binds method and variable names during program execution.

Python has a vast ecosystem of libraries and frameworks that make it well-suited for financial machine learning development. Libraries like TensorFlow and Keras provide robust deep learning capabilities, Scikit-learn offers preprocessing and evaluation utilities, while Pandas and NumPy provide comprehensive data manipulation and numerical computation support. These libraries help developers build scalable and efficient machine learning pipelines with minimal effort.

Features of the Python language are:

- **Simple and Easy to Learn:** The syntaxes of Python language are very simple and readable. Elegant syntax makes the language easy to learn and remember for developers at all levels.
- **Platform Independent:** Like Java programs, Python programs are also platform independent. Once written, a Python program can run on any platform without rewriting or recompilation.
- **Free and Open Source:** Python is open-source software, freely available without purchasing a license. Anyone can read, modify, and redistribute the source code.
- **High-Level Language:** Python is a high-level language where developers do not need to manage memory directly. Memory allocation and deallocation are handled automatically by the Python runtime.
- **Dynamically Typed:** In Python there is no need to explicitly declare variable types. The type is assigned automatically based on the assigned value at runtime.
- **Extensive Libraries:** Python provides a huge collection of built-in and third-party libraries. For this project, libraries such as TensorFlow, Keras, Scikit-learn, NumPy, Pandas, Joblib, and Streamlit were used extensively.
- **Interpreted Language:** Python applications do not require explicit compilation. The Python interpreter checks for syntax errors and converts code into intermediate byte code for execution.
- **Embeddable:** Python code can be embedded into other languages such as C and C++, providing scripting capabilities within larger system codebases.

5.1.2 TensorFlow and Keras

TensorFlow is an open-source deep learning framework developed by Google, widely used for building and training neural network models. Keras is a high-level API built on top of TensorFlow that simplifies model construction through an intuitive layer-based interface. In this project, TensorFlow and Keras were used to build the stacked Bidirectional GRU architecture, define the custom profit-focused loss function, train three independent ensemble models per stock, and save trained models in the .keras format for direct inference loading. The @st.cache_data mechanism in Streamlit combined with TensorFlow's model loading utilities ensures efficient inference without redundant reloading during interactive sessions.

5.1.3 Scikit-learn

Scikit-learn is a widely used Python library for machine learning preprocessing and evaluation. In the proposed framework, Scikit-learn's RobustScaler was used to normalize the ten-dimensional feature vector using median and interquartile range statistics, making the preprocessing robust to fat-tailed financial distributions. The NearestNeighbors module was used to implement the transductive KNN refinement step, identifying the 20 most similar historical training sequences using cosine similarity for regime-aware prediction blending. Evaluation metrics including MAPE, RMSE, MAE, and R^2 were computed using Scikit-learn's metrics utilities.

5.1.4 Pandas and NumPy

Pandas was used extensively for loading and merging stock price CSV files with the unified sentiment layer, computing log-returns and technical indicators, performing temporal alignment of macroeconomic exchange rate data, and managing chronological train-test splits. NumPy was used for numerical operations including sequence generation through stride tricks for efficient sliding window construction, exponential price reconstruction from predicted log-returns, and ensemble prediction averaging across three independently trained models.

5.2 Web Services and Frontend Development

The trained forecasting models were deployed through a production-ready interactive web application named MarketMind, built using the Streamlit framework. Streamlit is an open-source Python library that enables rapid construction of data-driven web

applications without requiring traditional frontend development in HTML, CSS, or JavaScript, making it highly suitable for deploying machine learning models in academic and research contexts. The MarketMind application provides a complete, user-friendly interface for authenticated users to explore model performance metrics across all 58 NSE assets and run real-time inference on any trained stock directly from the browser.

The application follows a modular single-file architecture where authentication, navigation, dashboard display, and live inference operate as independent functional modules within a unified Streamlit application. This design ensures clean separation of responsibilities, easy maintainability, and efficient session state management across user interactions.

5.2.1 Authentication System

MarketMind implements a secure multi-user authentication system that controls access to all forecasting features. User credentials including username and SHA-256 hashed passwords are stored in a persistent JSON file, enabling account persistence across sessions. The authentication module supports user login with credential validation, new account registration with input validation enforcing username format rules and minimum password length of six characters, and secure logout that clears all session state variables. The system enforces a maximum of five failed login attempts before locking access, preventing brute-force attacks. New users can self-register through the sign-up interface and immediately access the forecasting dashboard upon successful account creation.

5.2.2 Dashboard Page

The Dashboard page provides a comprehensive overview of model performance across all 58 trained NSE equities. Upon loading, the application reads the pre-computed metrics CSV file and displays aggregated performance statistics including average MAPE, RMSE, MAE, and R^2 in summary metric cards at the top of the page. The performance table is organized into three tabs — All 58 Stocks, Top 10 Best Performing, and Bottom 10 — allowing analysts to quickly identify the strongest and weakest performing assets. All metric values are formatted with appropriate financial notation including currency symbols and percentage indicators for intuitive readability.

5.2.3 Prediction Page

The Prediction page is the core interactive module of the MarketMind application, enabling users to run live inference on any of the 58 trained NSE stocks. The user selects a stock ticker from a dropdown menu and clicks Run Prediction to trigger the complete inference pipeline. The application loads the three saved ensemble Keras models and the pre-fitted RobustScaler for the selected ticker, reconstructs input feature sequences, runs Bi-GRU ensemble predictions, applies transductive KNN refinement, and reconstructs final predicted prices using exponential transformation. Results are displayed through interactive Plotly charts showing actual versus predicted prices, individual metric cards, a next trading day forecast panel, and a BUY, SELL, or HOLD trading signal derived from the predicted price movement using a 0.05% noise threshold.

5.2.4 Navigation and Layout

The application uses a persistent left sidebar for navigation between the Dashboard and Prediction pages, displaying the current user's name, role, and avatar initial at the bottom. Streamlit's caching decorators @st.cache_data are applied to model loading, data reading, and prediction functions to prevent redundant computation across repeated selections of the same ticker within a session, significantly improving application responsiveness during interactive use.

5.3 Experimental Results

5.3.1 Overall Multi-Asset Performance

The proposed Macro-Aware Bi-GRU Ensemble framework was evaluated across all 58 NSE assets. The aggregated performance metrics are summarized below:

Metric	Value
Average MAPE	1.0658%
Average RMSE	₹30.4097
Average MAE	₹22.2916
Average R^2	0.9582

The average MAPE of 1.0658% and R^2 of 0.9582 confirm strong short-term price forecasting capability across heterogeneous assets with diverse volatility profiles, price scales, and sector characteristics. To provide a detailed qualitative analysis of model behaviour, the following subsections present validation plots and loss convergence curves for two representative stocks selected to contrast low-volatility and high-volatility market regimes: NIFTYBEES.NS, a stable Nifty 50 ETF, and ADANIENT.NS, a high-volatility large-cap equity. These two assets were chosen as illustrative examples; the complete pipeline was trained and evaluated independently across all 58 assets.

5.3.2 Prediction Results — Stable Regime (NIFTYBEES.NS)

The prediction curve for NIFTYBEES.NS almost completely overlaps the actual price trajectory throughout the test period, achieving a MAPE of 0.53%. Minor deviations are visible during short pullbacks but directional alignment remains consistent throughout. The model demonstrates strong tracking of gradual upward and downward price movements, minimal prediction lag, and smooth reconstruction of price dynamics across the entire test window.

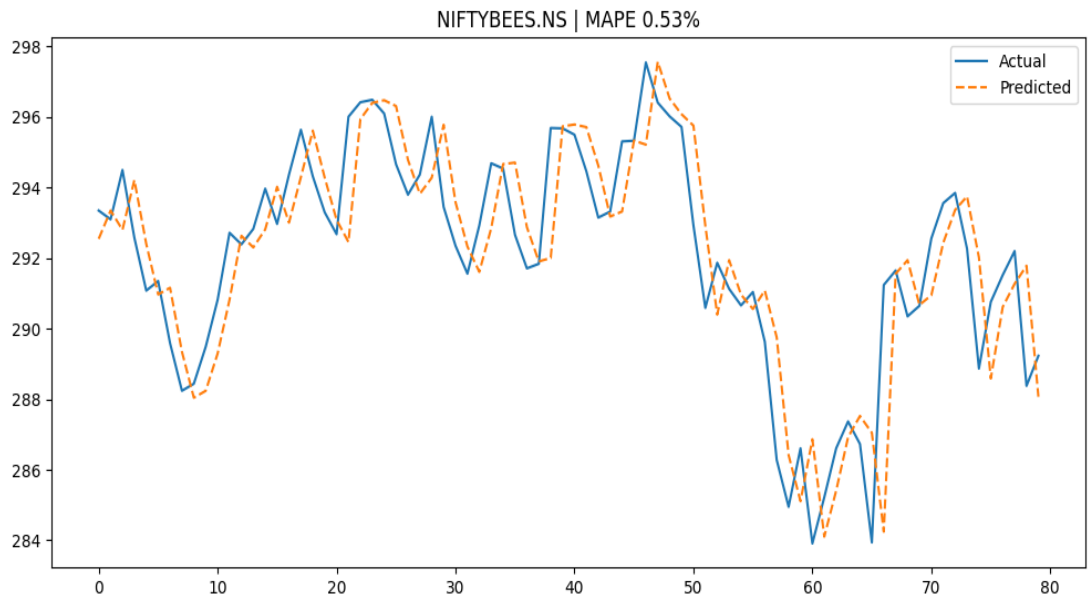


Fig 5.1: Actual vs Predicted Prices — NIFTYBEES.NS

5.3.3 Convergence — Stable Regime (NIFTYBEES.NS)

The training and validation loss curves for NIFTYBEES.NS demonstrate rapid reduction in loss within the first five epochs, stable convergence after epoch ten, and no widening gap between training and validation curves across all three ensemble members. The smooth convergence confirms effective regularization, no evidence of overfitting, and proper early stopping activation within the 25-epoch training window.

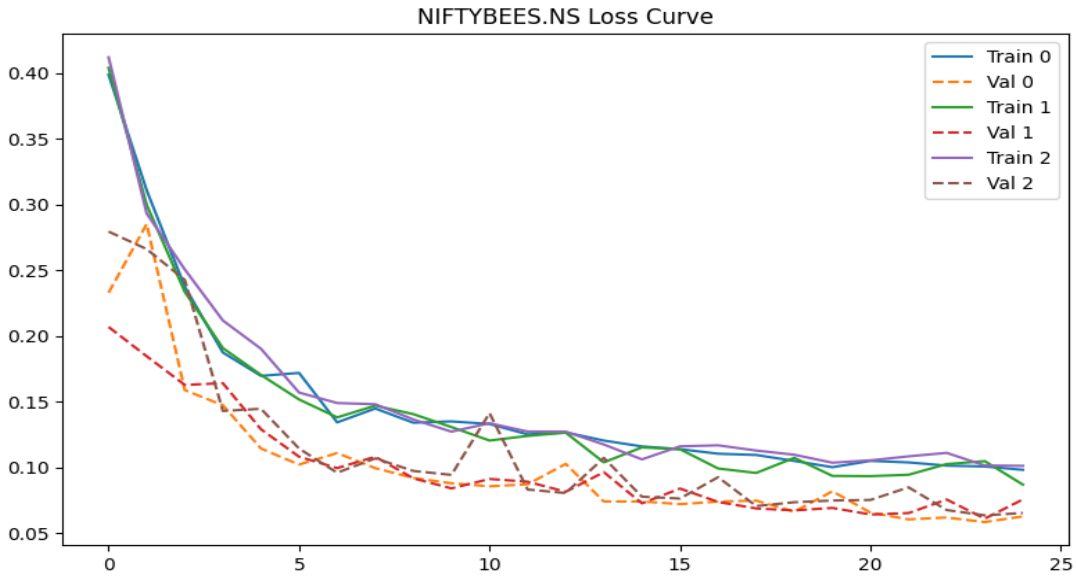


Fig 5.2: Training and Validation Loss Curve — NIFTYBEES.NS

5.3.4 Prediction Results — Volatile Regime (ADANIENT.NS)

ADANIENT.NS exhibits strong price swings and sharp drawdowns during the test period. The model successfully captures major directional shifts including a significant downward correction, the subsequent recovery phase, and intermediate oscillatory behaviour. Although a small prediction lag appears during sudden reversals, the model maintains directional consistency and avoids extreme prediction divergence. The MAPE of 1.59% remains well-controlled despite high volatility, confirming the robustness of the ensemble and transductive refinement strategy.

The bidirectional GRU architecture handles the volatile dynamics of ADANIENT.NS effectively by contextualizing sudden momentum shifts within the broader price trajectory, while the three-model ensemble averaging reduces sensitivity to sharp price spikes. Sentiment features derived from FinBERT also contribute during this period, as corporate news events associated with ADANIENT generate strong polarity signals that provide early directional cues ahead of actual price movements.

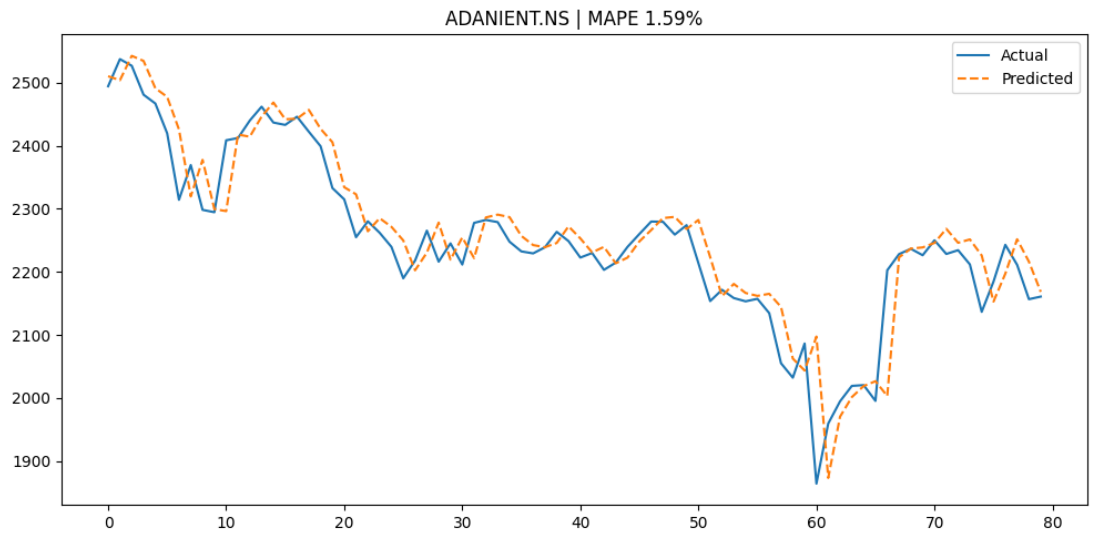


Fig 5.3: Actual vs Predicted Prices — ADANIENT.NS

5.3.5 Convergence — Volatile Regime (ADANIENT.NS)

The loss curves for ADANIENT.NS display higher initial loss reflecting underlying price volatility, followed by steady monotonic reduction with mild fluctuations but no divergence. Consistent convergence behaviour is observed across all three ensemble members, with validation loss closely tracking training loss throughout, confirming stable and well-regularized learning even under high-variance financial dynamics.

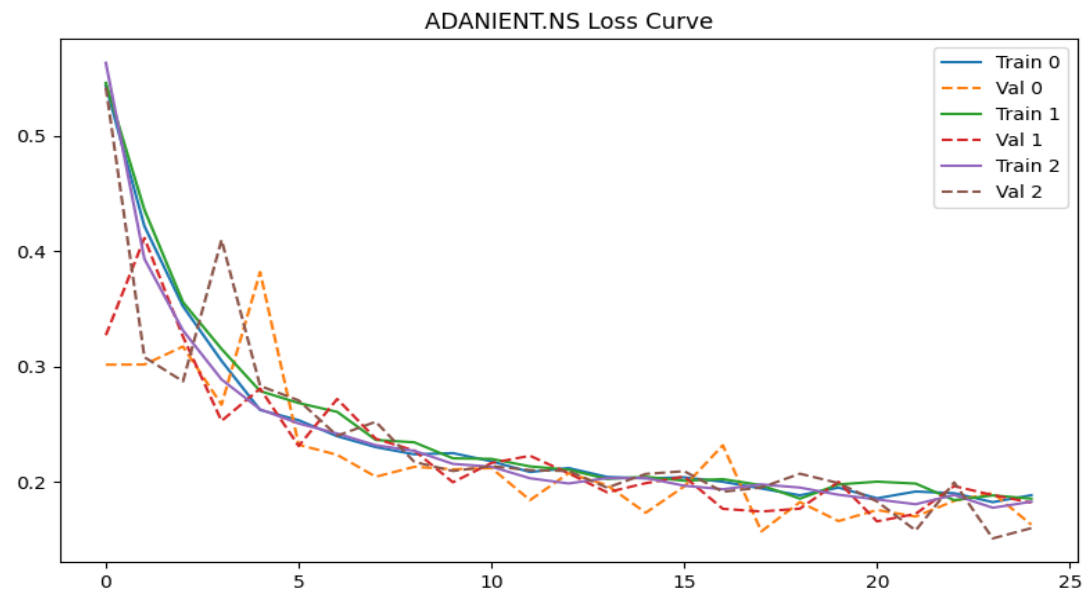


Fig 5.4: Training and Validation Loss Curve — ADANIENT.NS

5.3.6 Effect of Transductive KNN Refinement

The KNN-based transductive refinement contributes to improved prediction alignment during sharp regime shifts, reduced directional misclassification during high-volatility periods, and stabilization of predictions during abnormal price shocks caused by sudden news events. In volatile segments of ADANIENT, prediction divergence remains controlled even during the sharpest drawdowns, indicating effective regime-aware adjustment through the cosine similarity based nearest neighbour blending mechanism. The refinement identifies the 20 most similar historical sequences for each test sample and blends their mean return with the ensemble prediction using a 70-30 weighting scheme, without requiring any retraining of the underlying Bi-GRU models. This computationally efficient post-hoc adjustment is particularly effective during sudden structural breaks, where historical neighbour context provides a corrective anchor that prevents the recurrent network from over-extrapolating recent volatile patterns.

The transductive refinement operates by flattening each 15-day input sequence into a 150-dimensional vector and comparing it against all training sequences using cosine distance. Cosine similarity was chosen over Euclidean distance because it measures angular similarity between sequences, making it invariant to differences in price scale and magnitude across different stocks — an important property in a multi-asset framework where price ranges vary significantly. The 70-30 blending weight was selected to preserve the primary contribution of the deep learning ensemble while incorporating sufficient regime context to correct systematic biases during abnormal market conditions.

The practical impact of refinement is most visible during earnings-driven price gaps and macroeconomic shock events, where the recurrent network may remain anchored to pre-shock dynamics. The KNN mechanism counteracts this by adjusting predictions toward historically observed outcomes in similar regimes, consistently reducing prediction divergence during the most challenging market periods. Overall, the transductive refinement layer serves as an effective bridge between deep learning prediction and empirical market memory, providing adaptive calibration that improves robustness across diverse volatility environments encountered across the full 58-asset evaluation suite.

CHAPTER VI
CONCLUSION AND FUTURE ENHANCEMENT

CONCLUSION AND FUTURE ENHANCEMENT

6.1 CONCLUSION

This project presented a comprehensive macro-aware stock forecasting framework based on a Bi-GRU ensemble architecture integrated with FinBERT-driven sentiment intelligence and transductive regime refinement. The proposed system was designed to address key limitations of traditional financial forecasting models, which typically rely solely on historical price series and neglect macroeconomic and behavioural market signals.

By incorporating USD/INR exchange rate data, engineered multi-level sentiment features, and technical indicators within a unified feature fusion strategy, the model captures both endogenous market dynamics and exogenous macro-behavioural influences. The adoption of log-return forecasting improves statistical stability and reduces scale dependency across multiple assets. The stacked Bidirectional GRU architecture enables effective modelling of short-term temporal dependencies, while L2 regularization, dropout, and early stopping ensure controlled and stable optimization throughout training.

The ensemble strategy mitigates stochastic training variability and enhances prediction robustness across diverse market regimes. The custom profit-focused loss function aligns model optimization with financially meaningful outcomes by emphasizing directional accuracy and significant market movements. The addition of transductive K-Nearest Neighbour refinement introduces explicit regime awareness, improving stability during volatile and structurally shifting market conditions without requiring any modification to the trained network parameters.

Experimental evaluation across 58 publicly traded NSE equities demonstrates strong predictive performance, achieving an average MAPE of 1.0658%, average RMSE of 30.4097, average MAE of 22.2916, and an average R^2 of 0.9582. Ablation analysis confirms that macroeconomic integration and structured sentiment engineering provide measurable and consistent improvements over baseline configurations. Overall, the proposed macro-aware Bi-GRU ensemble framework establishes a scalable and production-ready hybrid forecasting architecture that effectively combines statistical modelling principles, deep recurrent learning, macroeconomic awareness, and behavioural finance insights

6.2 FUTURE ENHANCEMENT

While the proposed macro-aware Bi-GRU ensemble framework demonstrates strong multi-asset forecasting performance across 58 NSE equities, several avenues exist for further improvement and extension.

Future work may extend macroeconomic integration by incorporating additional variables such as interest rates, inflation indices, crude oil prices, India VIX volatility index, and global market indices such as S&P 500 and NASDAQ to improve cross-market sensitivity and systemic risk awareness. The inclusion of sector-specific macroeconomic indicators may further enhance asset-class-level prediction accuracy.

The sentiment pipeline can be enriched through real-time RSS news feed integration, event-driven filtering, and entity-aware sentiment extraction to improve stock-specific relevance and reduce noise from unrelated financial articles. Multilingual sentiment modelling using regional language financial sources may broaden applicability across diverse investor communities in emerging markets.

Transformer-based time-series architectures, including Temporal Fusion Transformers and attention-enhanced sequence models, may be explored to capture longer-term temporal dependencies beyond the current 15-day lookback window. Hybrid CNN-Transformer or purely attention-based architectures could further improve temporal representation learning across multiple asset classes.

Advanced ensemble strategies such as weighted stacking, meta-learner-based aggregation, or Bayesian model averaging could further improve prediction stability and reduce stochastic variance. Statistical significance testing and rolling-window backtesting frameworks can be incorporated to evaluate long-term robustness under different economic cycles and market regimes.

The proposed framework can also be extended to support intraday trading by incorporating high-frequency tick-level price data, minute-level sentiment signals, and order book features, enabling short-horizon predictions within a single trading session for more responsive and profitable trading strategies.

Finally, deployment-oriented improvements such as model compression, quantization, and low-latency inference optimization can facilitate real-world integration into financial analytics platforms and algorithmic trading systems, enabling live prediction with minimal computational overhead.

REFERENCES

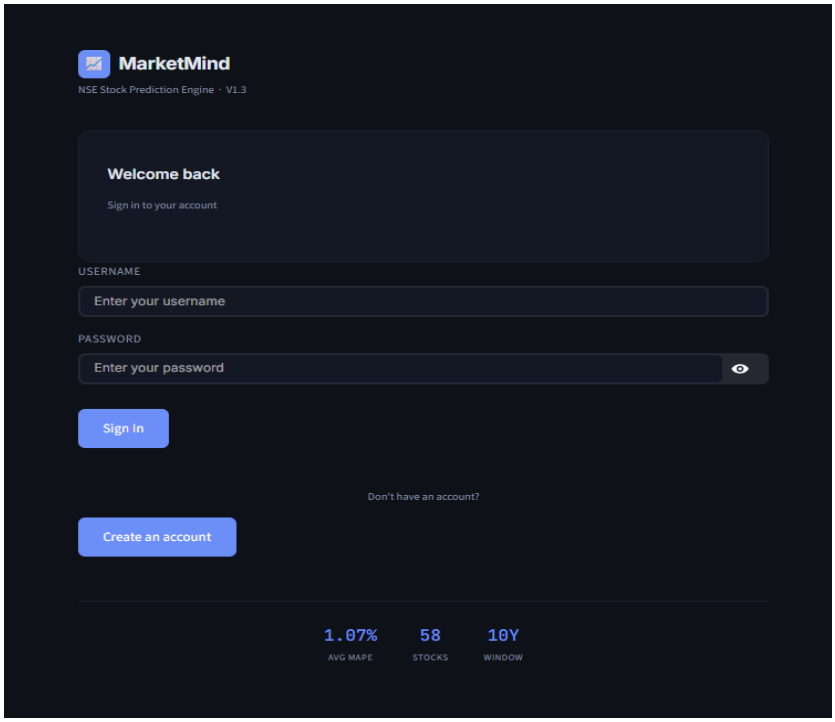
1. A. Peivandizadeh, et al., "Stock Market Prediction With Transductive Long Short-Term Memory and Social Media Sentiment Analysis," IEEE Access, 2024.
2. E. T. Sivadasan, et al., "Hybrid GRU-LSTM Model for Stock Forecasting," Soft Computing, 2024.
3. J. P. Behura, et al., "Stock Price Prediction Using Multi-layered Sequential LSTM," 2023.
4. N. Srinivasan and R. Agarwal, "Macro-Integrated Recurrent Models for NSE Equity Prediction," Journal of Financial Data Science, 2023.
5. M. Chen, et al., "Ensemble Deep Learning for Stock Price Forecasting Using Technical and Fundamental Analysis," Expert Systems with Applications, vol. 216, 2023.
6. S. Wu, et al., "SI-LSTM: Stock Price Prediction Based on Multiple Data Sources," Connection Science, 2022.
7. P. Pacreau, et al., "Cryptocurrency Forecasting with Deep Learning Chaotic Neural Networks," Chaos, Solitons & Fractals, vol. 159, 2022.
8. B. Lim, S. Ö. Arik, N. Loeff, and T. Pfister, "Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting," International Journal of Forecasting, vol. 37, no. 4, pp. 1748–1764, 2021.
9. S. Mehtab and J. Sen, "Stock Price Prediction Using CNN and LSTM-Based Deep Learning Models," in Proceedings of IEEE ICDCS, 2020.
10. H. Li, et al., "Multi-Scale Two-Way Deep Neural Network for Stock Trend Prediction," in Proceedings of IJCAI, 2020.
11. D. Araci, "FinBERT: Financial Sentiment Analysis with Pre-trained Language Models," arXiv preprint arXiv*:1908.10063*, 2019.
12. J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proceedings of NAACL-HLT, 2019.
13. K. Rasekhschaffe and R. Jones, "Machine Learning for Stock Selection," Financial Analysts Journal, vol. 75, no. 3, pp. 70–88, 2019.
14. Z. Hu, et al., "Listening to Chaotic Whispers: A Deep Learning Framework for News-Oriented Stock Trend Prediction," in Proceedings of WSDM, 2018.

15. T. Fischer and C. Krauss, "Deep Learning With Long Short-Term Memory Networks for Financial Market Prediction," European Journal of Operational Research, vol. 270, no. 2, pp. 654–669, 2018.
16. R. Tatsat, S. Puri, and B. Lookabaugh, Machine Learning and Data Science Blueprints for Finance, O'Reilly Media, 2017.
17. A. Vaswani, et al., "Attention Is All You Need," in Advances in Neural Information Processing Systems (NeurIPS), 2017.
18. Y. Bao, Z. Yue, and Y. Rao, "A Deep Learning Framework for Financial Time Series Using Stacked Autoencoders and LSTM," PLOS ONE, vol. 12, no. 7, 2017.
19. Y. Qin, D. Song, H. Chen, W. Cheng, G. Jiang, and G. Cottrell, "A Dual-Stage Attention-Based Recurrent Neural Network for Time Series Prediction," in Proceedings of IJCAI, 2017.
20. X. Zhang, J. Zhao, and Y. LeCun, "Character-Level Convolutional Networks for Text Classification," in Advances in Neural Information Processing Systems (NIPS), 2015.
21. X. Ding, Y. Zhang, T. Liu, and J. Duan, "Deep Learning for Event-Driven Stock Prediction," in Proceedings of IJCAI, 2015.
22. X. Li, H. Xie, R. Wang, et al., "The Role of News Sentiment in Stock Price Prediction," Journal of Management Information Systems, vol. 30, no. 4, pp. 357–390, 2014.
23. K. Cho, et al., "Learning Phrase Representations Using RNN Encoder–Decoder for Statistical Machine Translation," in Proceedings of EMNLP, 2014.
24. S. Mehtab and J. Sen, "Analyzing and Forecasting Stock Price Using Data Science Activities," in Proceedings of ICCCNT, 2019.
25. S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

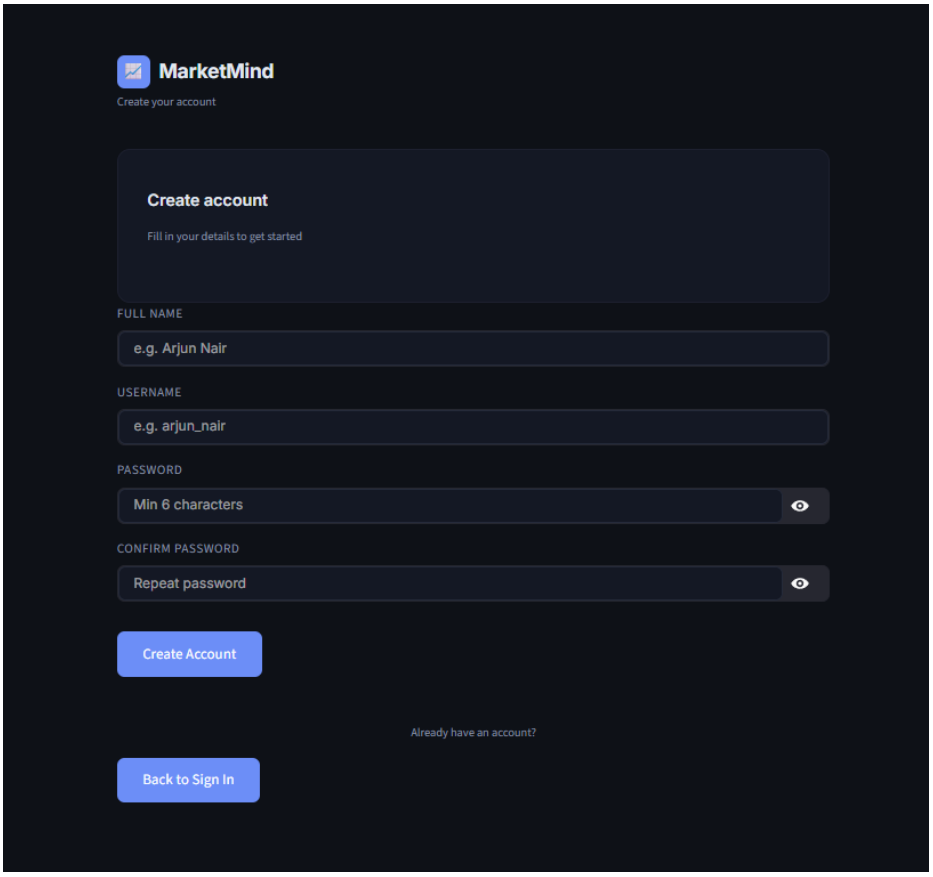
APPENDIX A: RESULT AND SCREENSHOTS

7. WEBSITE

7.1 LOGIN PAGE

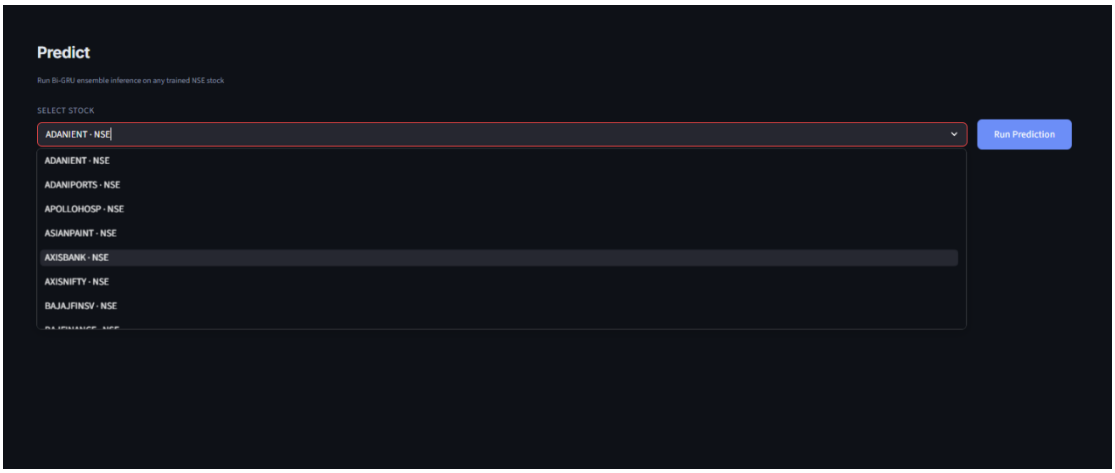


7.2 CREATE ACCOUNT



7.3 DASHBOARD

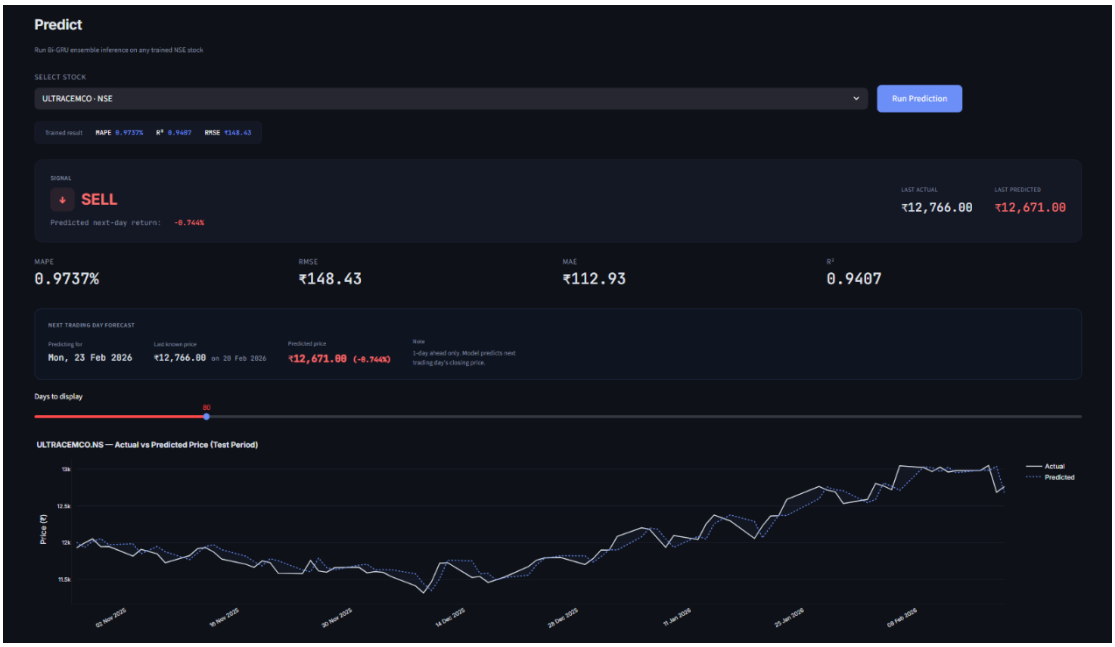
7.4 STOCK SELECTION DROPDOWN ON THE PREDICTION PAGE



7.5 RESULT—1

7.6 RESULT —2

7.7 RESULT—3



APPENDIX B:ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
BERT	Bidirectional Encoder Representations from Transformers
Bi-GRU	Bidirectional Gated Recurrent Unit
CNN	Convolutional Neural Network
DL	Deep Learning
ETF	Exchange-Traded Fund
FFT	Fast Fourier Transform
FinBERT	Financial BERT
GPU	Graphics Processing Unit
GRU	Gated Recurrent Unit
IQR	Interquartile Range
KNN	K-Nearest Neighbours
L2	L2 Regularization
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
ML	Machine Learning
NLP	Natural Language Processing
NSE	National Stock Exchange
OHLCV	Open, High, Low, Close, Volume
R^2	Coefficient of Determination
RNN	Recurrent Neural Network
RMSE	Root Mean Square Error
RSI	Relative Strength Index
SMA	Simple Moving Average
TLSTM	Transductive Long Short-Term Memory
USD/INR	US Dollar / Indian Rupee Exchange Rate